



Report

Chapter 1. Application Design

1.1 Core Features

1.2 User Interface

1.3 Implications for the Data Model

Chapter 2. Database Rework

2.1 Observed Issues

2.1 Name Changes

2.2 Table Redesign

2.2.1 Normalization

2.2.2 Key Analysis

Chapter 3. Design

3.1 Movie Data Model

Titles and Episodes

Genres

Ratings

Alternative Titles (Also Known As)

People and Professions

Cast and Crew

3.2 Framework Model

Account-Centered Design

Search History

Rating History

Bookmarks	
Integration with the Movie Data Model	
3.3 Reverse-engineered ERD	
Chapter 4. Implementation	
4.1 Tables	
4.1.1 movie_db schema (content metadata)	
4.1.2 profile schema (user data)	
Functionality	
1-D Functionality (Implemented API)	
1-D.1 Basic framework functionality	
1-D.2 Simple search	
1-D.3 Title rating	
1-D.4 Structured string search	
1-D.5 Finding names	
1-D.6 Finding co-players	
1-D.7 Name rating	
1-D.8 Popular actors	
1-D.9 Similar movies	
1-D.10 Frequent person words: person_words(person_name [, max_words])	
1-D.11-13	
Chapter 5. Data Migration	
Chapter 6. Indexing	
Chapter 7. Testing	
Chapter 8. Deployment	
Restore from backup	
4.5 Materialized Views	
4.4 Views	
4.3 Stored Procedures	

<div style="page-break-before: always;"></div>

Chapter 1. Application Design

1.1 Core Features

The application is designed to support three primary user needs:

1. **Exploration of movie data:** searching titles, browsing episodes, inspecting ratings and alternative names.
2. **Personal engagement:** rating movies, bookmarking items, tracking search history.
3. **Knowledge discovery:** analyzing relationships between actors, co-players, and genres.

To support these needs, the database is split into three cooperating schemas:

- `movie_db` → authoritative metadata about titles, people, ratings, episodes, and genres.
- `profile` → account-centered data such as search history, rating history, and bookmarks.
- `api` → a functional abstraction layer providing controlled access (functions, procedures, materialized views).

This design enforces a separation between user-generated data and static metadata. It allows parallel growth of both without coupling, and supports later database separation.

1.2 User Interface

User interaction centers on searching, browsing, and contributing ratings or bookmarks.

- **Dashboard:** Centralized search bar and navigation drawer, emphasizing bookmarks and search as most-used features.
- **Search Results:** Compact, scannable presentation (poster, title, year, short plot).

- **Detail Views:** Rich metadata with linked navigation between titles and people; actions to bookmark or rate directly.
- **User Profile:** Access to bookmarks and ratings, reflecting the account-centered design.
- **Bookmark and Rating History Views:** Dedicated displays with potential for future sorting/filtering.

These workflows mirror expectations from comparable platforms and directly map to database features.

1.3 Implications for the Data Model

Supporting account-centered features requires more than movie and person metadata. Users must persist their own state — bookmarks, searches, and ratings — which requires additional tables and clear separation from canonical film data.

Key principles:

- **API contracts:** Clients never access tables directly; they call API functions or views. This allows the underlying schema to evolve without breaking clients.
- **No cross-boundary mixing:** Each schema owns its data. Updates that affect both `profile` and `movie_db` are two API calls, orchestrated by the backend.
- **No joins across boundaries:** The backend combines data; the database does not.
- **Function evolution:** New versions (e.g., `api.add_rating_v2`) can be added while old ones remain for compatibility.

This adds some orchestration complexity in the backend but ensures clear ownership, maintainability, and resilience. The model avoids tight coupling and remains adaptable to future growth.

Chapter 2. Database Rework

2.1 Observed Issues

Looking at the legacy schema, several recurring problems stand out

- **Denormalized fields:** Lists of genres, directors, writers, professions, and “known for” titles are stored as comma-separated strings, which are hard to query and validate.
- **Weak typing:** IDs stored as fixed-length strings (`char(10)`), numerics embedded in text, and no constraints on common fields (years, seasons, episodes).
- **Inconsistent and opaque naming:** Identifiers such as `tconst` , `nconst` , and table names like `wi` rely on short aliases rather than descriptive terms. Column naming mixes camelCase and snake_case. These practices reduce readability, increase onboarding time, and make the schema harder to maintain.
- **Coupled enrichment:** `omdb_data` mixes attributes like awards, runtime, and ratings, some overlapping with existing tables, leading to redundancy.

2.1 Name Changes

A major issue in the legacy schema is the **naming convention**. Identifiers often use short aliases (`tconst` , `nconst` , `wi`) that are not self-explanatory. This reduces readability and requires external documentation to understand.

Problems observed:

- **Opaque identifiers** – `tconst` and `nconst` do not indicate titles or people.
- **Abbreviations** – short names like `wi` and fields such as `isAdult` mix casing and lack clarity.
- **Inconsistent conventions** – camelCase and snake_case appear in the same schema; column capitalization is inconsistent.

Redesign principles:

- **Primary keys:** always named `id` .

- **Foreign keys:** named after the referenced table (`title_id` , `person_id`).
- **Tables:** descriptive, singular, snake_case (`title` , `person` , `actor` , `crew` , `word_index`).
- **Booleans:** prefixed with `is_` (e.g., `is_adult`).
- **Timestamps:** suffixed with `_at` (e.g., `created_at`).

This consistent convention improves readability, makes queries self-explanatory, and reduces the risk of misinterpretation.

2.2 Table Redesign

The legacy schema stored relationships such as genres, directors, writers, professions, and “known for” titles as comma-separated strings.

This introduced several problems:

- **Unqueryable values** – filtering required string parsing instead of joins.
- **No referential integrity** – no guarantee that referenced titles or people existed.
- **Redundant storage** – values repeated across rows, causing inconsistency.
- **Difficult maintenance** – updating a single attribute required rewriting multiple rows.

2.2.1 Normalization

The redesign applies **3rd Normal Form (3NF)** to eliminate repeating groups and ensure each fact is stored once. Complexity moves from string parsing to relational joins, which databases handle efficiently.

- **Join tables** replace string lists:
 - `movie_db.title_genre` links titles to genres.
 - `movie_db.title_director` , `movie_db.title_writer` , `movie_db.actor` , and `movie_db.crew` capture people-to-title relationships.

- `movie_db.person_profession` and `movie_db.person_known_for` normalize person attributes.
- **Controlled vocabularies** define common categories (genres, professions) via reference tables or enums.
- **Consistent datatypes** ensure clarity and safety: UUIDs for identifiers, booleans prefixed with `is_`, timestamps suffixed with `_at`.

2.2.2 Key Analysis

In the legacy schema, `tconst` and `nconst` served as primary keys stored as `VARCHAR(20)` to align with IMDb/OMDb datasets and the custom word index (`wi`). While this simplified ingestion, it caused inefficiency in joins, lacked uniqueness guarantees, and tightly coupled the schema to external formats.

Redesign

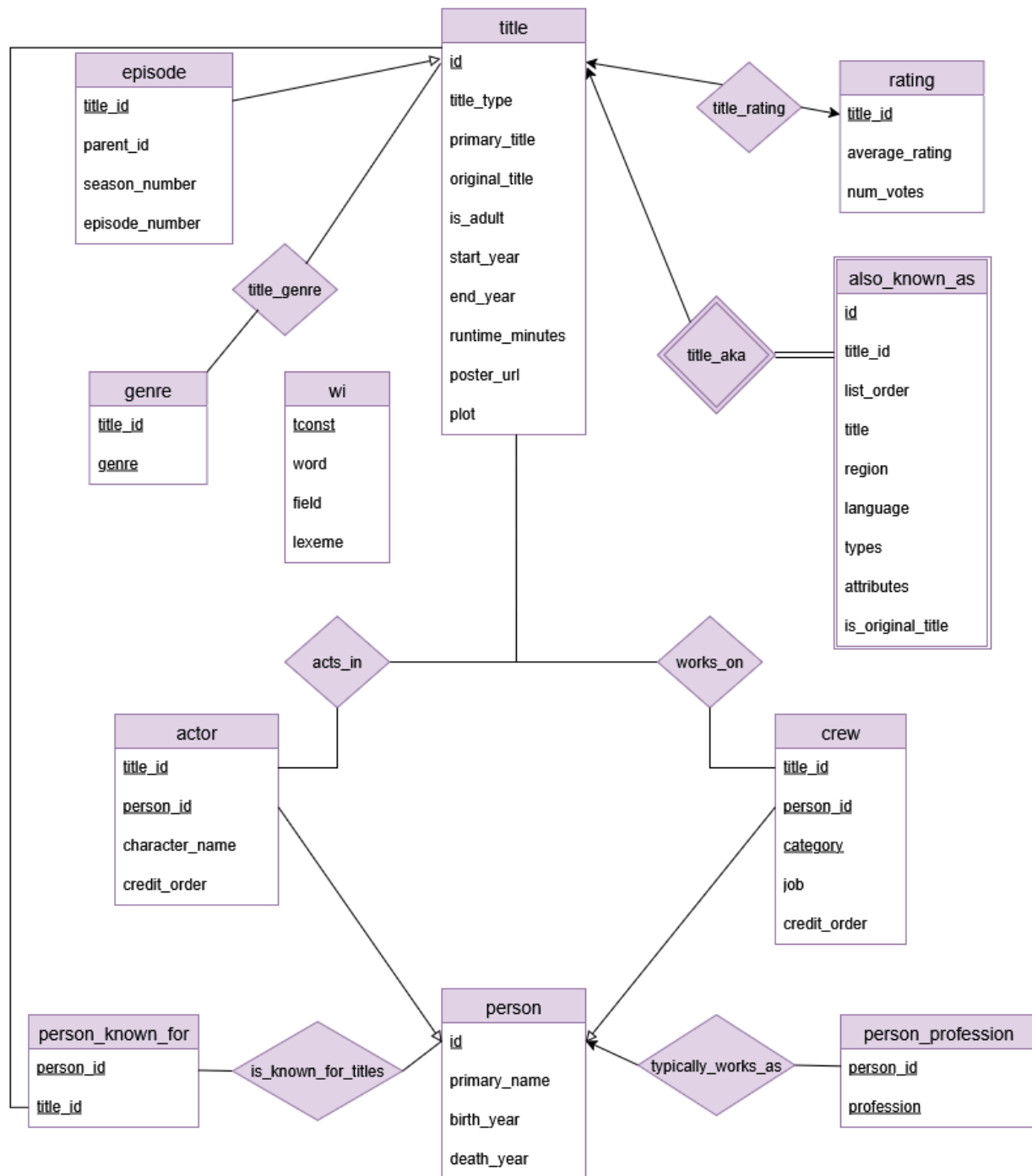
- All entities use **UUIDs** as primary keys.
- **Foreign keys** enforce referential integrity across join tables.
- External identifiers (e.g., IMDb `legacy_id`) are retained only as metadata, not as keys.

UUIDs provide globally unique and system-controlled identifiers, independent of external systems. Foreign keys guarantee valid relationships, while legacy IDs remain available for interoperability. This improves **integrity, performance, and maintainability** without losing compatibility with external datasets.

Chapter 3. Design

3.1 Movie Data Model

The Movie Data Model provides the structural foundation for representing titles, people who worked on those titles, and their relationships.



1-B Movie Data Model - ER Diagram

Titles and Episodes

Titles are central to the model. Since each title has a unique plot and poster, these attributes are stored directly in the title table to simplify queries. Specialized cases

such as episodes are treated as **partial specializations of titles**, with additional attributes (e.g., `parent_id`) stored in a dedicated table. This approach avoids overcomplicating the schema while retaining essential hierarchical information.

Genres

Genres are normalized into a separate table because titles can belong to multiple genres. The many-to-many relationship between titles and genres prevents redundancy and allows efficient retrieval of genre-based queries, which are frequent in user-driven searches.

Ratings

A dedicated rating table maintains the current average rating and number of votes per title. This ensures that aggregation is efficient and that global ratings remain synchronized with user contributions.

Alternative Titles (Also Known As)

The *also_known_as* entity is modeled as a weak entity dependent on titles, since it cannot exist on its own. Each alias must reference a title, ensuring total participation in the relationship. Constraints ensure consistency between the alias and the original title, preserving data integrity.

People and Professions

The person entity is normalized by splitting “known for” titles and professions into separate many-to-many relationships. This prevents storage of comma-separated lists and allows efficient querying.

A person may be associated with multiple titles and professions, while each title or profession may also relate to multiple people.

Cast and Crew

Actors and crew are modeled as **specializations of person**, since they require role-specific attributes. Actors are separated from crew to reflect their distinct relevance to users and to eliminate unnecessary null values (e.g., character fields that do not apply to crew).

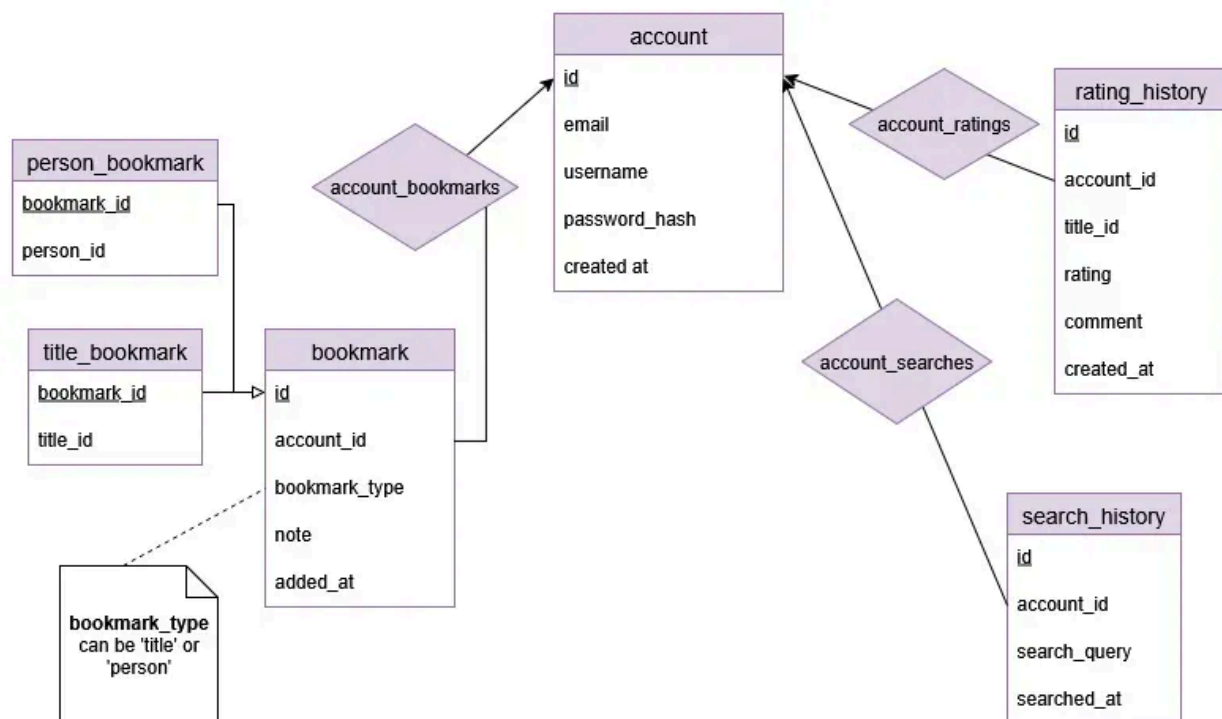
Relationships to titles are represented as many-to-many (e.g., "acts_in" and "works_on"). Ordering information is preserved for actors to reflect their prominence in credits.

The original *title_crew* table from IMDB was excluded because its information is redundant with other sources (e.g., *title_principals*). This decision reduces duplication while retaining all necessary information.

We found the *category* and *job* attributes in the original *title_principals* to be too inconsistent in their naming to be mapped in a separate table. Ideally, a function could have been written to streamline this person metadata but we decided against it not to overcomplicate our design. Perhaps this is something that could be improved upon later in the project.

3.2 Framework Model

The Framework Model provides the structures necessary to support user-facing features and complements the Movie Data Model.



1-C Framework Model - ER Diagram

Account-Centered Design

At the core of the Framework Model is the account entity. Most framework tables — `search_history`, `rating_history`, and `bookmark` — are linked to accounts in a many-to-one relationship, ensuring that user activity is consistently tracked.

Search History

The search history table logs all user queries with timestamps. This enables transparency for the user and establishes a reliable dataset for potential analytics or personalization features.

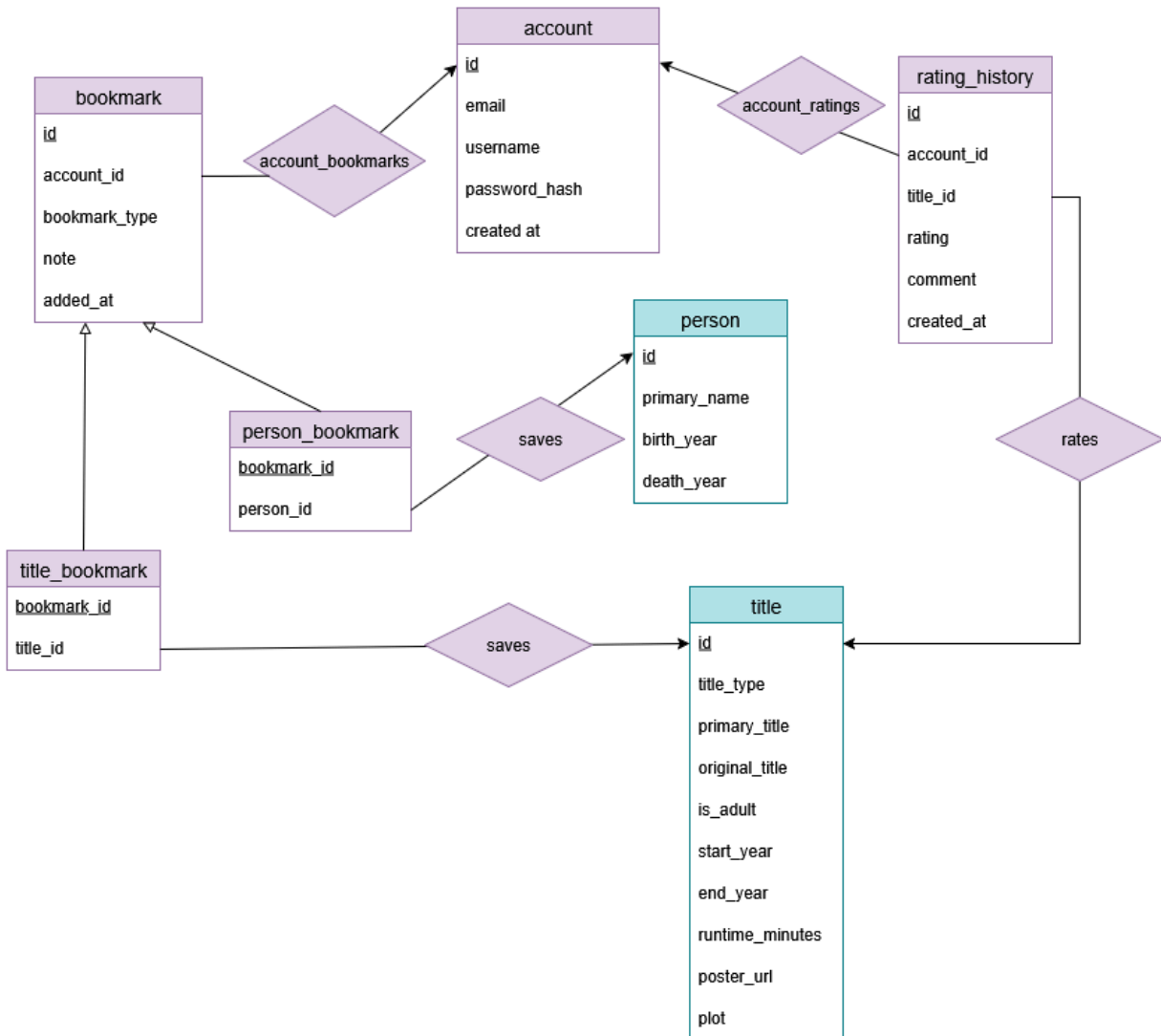
Rating History

Each rating action is stored in the rating history table with timestamps. Even if a user updates a rating, prior ratings should remain in the database. This approach preserves historical accuracy while ensuring that only the most recent rating contributes to the title's global average.

Bookmarks

The bookmark entity is generalized to accommodate both people and titles. Subtypes (`person_bookmark` and `title_bookmark`) store specific references. A type indicator ensures clarity while avoiding schema duplication.

Integration with the Movie Data Model



1-C Framework Model integrated with the Movie Data Model - ER Diagram

Framework entities reference Movie Data Model entities through identifiers (`title_id` , `person_id`). These relationships are conceptual rather than enforced by strict foreign keys, preserving separation between the two models.

3.3 Reverse-engineered ERD

Chapter 4. Implementation

4.1 Tables

The database is organized into two main schemas:

- `movie_db` – normalized metadata about titles, people, and their relationships.
- `profile` – user accounts, interactions, and personal activity history.

All entities use **UUID primary keys** for global uniqueness. IMDb identifiers are preserved as `legacy_id` for interoperability but are no longer primary keys.

4.1.1 movie_db schema (content metadata)

- `title`
Core table for all films, shows, and episodes. Stores identifiers, titles, adult flag, release years, runtime, poster, and plot.
- `user_rating`
One rating per `(account, title)`. Ensures each user has at most one active rating for a title. Used to calculate aggregates.
- `rating`
Maintains aggregated statistics: average rating and number of votes per title. Derived from `user_rating` for fast lookups.
- `genre`
Join table linking titles to one or more genres. Replaces comma-separated strings with one row per `(title, genre)`.
- `episode`
Represents episodic structure. Links episodes to parent series (`parent_id`) and records season/episode numbers.
- `also_known_as`
Alternative titles (regional, language-specific, or descriptive variations). Normalized from denormalized aliases.

- **person**
Stores people (actors, directors, writers, crew). Contains legacy IMDb IDs, names, and life dates.
- **person_known_for**
Many-to-many join between people and their "known for" titles.
- **person_profession**
Join table linking people to professions (e.g., "actor", "director"). Normalizes multi-valued legacy field.
- **crew**
Assigns people to titles with roles, categories, and credit order. Covers directors, writers, and crew not in the actor table.
- **actor**
Specialized table for cast members. Links titles to persons with character names and credit order.
- **word_index**
Inverted index for search and analytics. Stores tokenized words from titles/plots with optional lexemes. Used for keyword queries and co-occurrence analysis.

4.1.2 profile schema (user data)

- **account**
User account data with email, username, password hash, and creation timestamp.
- **bookmark**
Unified bookmark system. Each row links a user to either a title or a person, enforced with the **bookmark_target** enum. Optional JSON notes can be attached.
- **search_history**
Tracks user search queries with timestamps. Used for personalization and analytics.
- **rating_history**

Stores a historical record of user ratings with optional comments. Unlike `movie_db.user_rating`, this table preserves history for audit and UI purposes.

Functionality

1-D Functionality (Implemented API)

All functions are implemented on top of our redesigned Movie Data Model and our Framework Model created and loaded by our repeatable build scripts.

1-D.1 Basic framework functionality

Goal.

Support users, bookmarks, notes, and history as the foundation for later features.

Design & Implementation.

- Users. `create_user`, `find_user`, `change_password`, `delete_user` manage profile rows. We keep the interface minimal and deterministic (return id on create; void for updates/deletes).
- Bookmarks. Unified bookmark table supports *either* a title or a person per row, enforced with a CHECK constraint; functions `add/remove/list` both kinds.
- Notes (extension). Two small tables (`note_title`, `note_person`) store user-authored text; CRUD functions `upsert` and `fetch` notes with `updated_at`.
- History. Read-only accessors `get_search_history(profile_id, limit)` and `get_rating_history(profile_id, limit)` provide paginated, time-ordered results and join titles for readability.

Why this design.

The single bookmark table plus constraints avoids duplication and simplifies uniqueness rules per (user, object). Notes are an allowed framework extension.

1-D.2 Simple search

Function.

`api.string_search_title(p_query TEXT)`

Design.

Full-text search with `to_tsvector('english', primary_title || ' ' || coalesce(plot, ''))`
`@@ plainto_tsquery('english', p_query)`, ranked by `ts_rank`. (Substring fallback via trigram indexes exists but is not the default.)

Outcome.

Fast, ranked results; language-aware matching (stemming/lexemes).

1-D.3 Title rating

Functions.

`api.add_user_title_rating(account_id UUID, title_id UUID, rate INT)` updates `movie_db.user_rating` (one row per user/title) and recomputes `movie_db.rating` (avg + count). `api.add_rating(...)` writes/updates `profile.rating_history` for audit/UI.

Policy.

Overwrite per (account, title). Aggregates are derived from `user_rating`, not from `rating_history`.

1-D.4 Structured string search

Function.

`api.structured_string_search(title_q, plot_q, char_q, person_q)`

Design.

LEFT JOIN `movie_db.actor a` and `movie_db.person p`; ILIKE filters per non-NULL parameter; distinct title rows.

1-D.5 Finding names

Goal.

Provide direct lookup of persons (e.g., actors) by substring.

Design & Implementation.

Function. `api.get_person_by_name(search_term TEXT) → (person_id, name)` from `movie_db.person` via `ILIKE`.

Outcome.

Small, composable building block for UI autosuggest and for 1-D.6.

1-D.6 Finding co-players

Goal.

Given an actor's name, list the most frequent co-players with counts.

Function.

`api.get_coplayers(p_actor_name TEXT)`

Design.

Resolve actor → titles → co-occurring actors with frequency; join back to names.

Outcome.

Simple co-occurrence analytics based on our normalized actor table; easy to wrap in a view if desired for readability.

1-D.7 Name rating

Goal.

Derive a popularity score for names from the ratings of related titles, weighted by `num_votes`.

Design & Implementation.

Object. `movie_db.get_actor_avg_title_rating` (materialized view)

Design.

Weighted average over an actor's filmography:

$\text{SUM}(\text{avg_rating} * \text{num_votes}) / \text{SUM}(\text{num_votes})$ from `movie_db.rating`.

Outcome.

Materialized, query-friendly ratings for names used by 1-D.8.

1-D.8 Popular actors

Goal. Utilize name ratings to rank actors relevant to a context (we implement the *co-players of a given actor, ranked by popularity* option).

Function.

`api.get_popular_co_actors(p_actor_name TEXT)`

Design.

Co-players of the target actor ordered by
`get_actor_avg_title_rating.weighted_rating (NULLS LAST)`, then name.

Outcome.

A user-facing list that privileges widely-voted, well-rated collaborators.

1-D.9 Similar movies

Goal.

Provide a principled notion of similarity and list comparable titles.

Function.

`api.get_similar_movies(p_title_id UUID, p_limit INT=20)`

Design.

Jaccard similarity over `movie_db.genre`.

Outcome.

Deterministic, scalable baseline similarity suitable for UI “More like this” sections.

1-D.10 Frequent person words: `person_words(person_name [, max_words])`

Goal.

Characterize a person by frequent words across titles they are involved in, using the provided inverted index wi.

Function.

api.person_words(p_person_name TEXT, max_words INT=10)

Design.

person → actor → movie_db.word_index; group by word, filter short tokens; return top-K.

Outcome. Lightweight descriptive tags usable in UIs (e.g., word clouds).

1-D.11-13

Tables.

Use movie_db.word_index (not wi).

api.exact_match_query(keywords TEXT[]) — ALL keywords (case-insensitive).

api.best_match_query(keywords TEXT[]) — rank by COUNT(DISTINCT word).

api.word_to_words_query(p_keywords TEXT[], p_limit INT=20) — co-occurring words ranked by frequency.

Chapter 5. Data Migration

The migration script transfer data from the legacy IMDb-style tables into the redesigned `movie_db` schema while applying normalization, type corrections, and referential integrity.

Process:

1. **Word index** – Legacy `wi` entries are migrated into `movie_db.word_index`, mapping legacy `tconst` values to new UUID-based `title.id`. `TRIM` is applied to remove fixed-length CHAR padding.
2. **Titles** – Records from `title_basics` and extended attributes from `omdb_data` are merged into `movie_db.title`. Fields such as `start_year`, `end_year`, and `runtime_minutes` are cast to integers with `NULLIF` guards for missing values.

3. **Episodes** – Hierarchical relationships from `title_episode` are migrated into `movie_db.episode`, linking each episode to both its own title UUID and its parent series UUID.
4. **Genres** – The comma-separated `genres` field is split into multiple rows in `movie_db.genre` using `string_to_array`, ensuring each (title, genre) pair is normalized.
5. **Alternate titles** – Data from `title_akas` is moved into `movie_db.also_known_as`, preserving ordering, region, language, and flags.
6. **Ratings** – Ratings are transferred from `title_ratings` into `movie_db.rating` with integrity checks applied.
7. **Persons** – Records from `name_basics` are migrated into `movie_db.person`, normalizing `birth_year` and `death_year`.
8. **Known-for titles** – Comma-separated `knownForTitles` values are expanded into rows in `movie_db.person_known_for`, with each entry validated against existing `title.id`.
9. **Professions** – Multi-valued `primaryProfession` entries are normalized into `movie_db.person_profession`.
10. **Crew and actors** – `title_principals` is split into two tables:
 - Non-actors → `movie_db.crew`.
 - Actors, actresses, self-roles → `movie_db.actor`.

Post-migration steps:

- Legacy source tables (`title_basics`, `name_basics`, `title_ratings`, etc.) are dropped once data is confirmed migrated.
- Verification queries confirm row counts for titles, episodes, genres, ratings, persons, actors, and crew.

The migration process ensures:

- Legacy identifiers (`tconst`, `nconst`) are preserved as `legacy_id` attributes for interoperability.

- Relationships are converted from comma-separated lists to normalized join tables.
- Datatypes are corrected (e.g., years as `INT`, UUIDs as PKs).
- Referential integrity is enforced through foreign keys.

This step transforms the dataset from a bulk-ingestion format into a fully relational structure aligned with the new design, while preserving compatibility with legacy IDs where required.

Chapter 6. Indexing

Efficient query performance requires careful use of indexes. In this design, indexes are added selectively to support common access patterns: search, joins, and time-ordered lookups. Both B-tree and GIN indexes are used, depending on the query type.

Title Search

- **Trigram indexes:**
 - `idx_title_primary_title_trgm` and `idx_title_plot_trgm` accelerate substring and ILIKE searches on titles and plots.
- **Full-text index:**
 - `idx_title_fulltext` supports language-aware searching via `to_tsvector`.
- **Use case:** enables fast ranked title lookups for both substring and semantic search.

User Ratings:

- `idx_user_rating_account` optimizes queries by `account_id`, e.g. listing a user's ratings.

Genre:

- `idx_genre_title` speeds joins from titles to their genres.
- `idx_genre_genre` accelerates filtering by genre.

Episodes:

- `idx_episode_parent` optimizes lookups of episodes by series (`parent_id`).
- `idx_episode_season` supports navigation by season and episode numbers.

Alternate Titles:

- `idx_aka_titleid` supports joins from titles to alternate names.
- `idx_aka_title_trgm` enables fast substring search on alternate titles.

People:

- `idx_person_name_trgm` accelerates substring search on person names.
- `idx_person_known_for_title` and `idx_person_profession` support queries by known-for titles and profession categories.

Actor and Crew:

- `idx_actor_title` , `idx_actor_person` , `idx_crew_title` , `idx_crew_person` speed joins between people and titles in both directions.

Word Index:

- `idx_word_index_word` and `idx_word_index_title` support keyword lookups and joins with titles.
- `idx_word_index_word_trgm` (optional) provides fuzzy search on words.

Profile Schema:

- **Bookmarks:** `idx_bookmark_account_type_added` enables fast lookups of a user's bookmarks, ordered by recency.
- **Search history:**
 - `idx_search_history_account_time` supports chronological history queries.
 - `idx_search_history_query_trgm` allows fast substring search on past queries.
- **Rating history:**
 - `idx_rating_history_title` links histories to titles.
 - `idx_rating_history_account_time` enables time-ordered user rating history retrieval.

Chapter 7. Testing

Testing is automated using SQL scripts executed inside transactions. By enabling `ON_ERROR_STOP`, any failing statement aborts the test immediately. Wrapping each test script in a `BEGIN ... ROLLBACK` block ensures that test data does not persist in the database.

Two categories of tests are included:

1. Profile and movie database tests

These scripts validate core functionality of the `profile` and `movie_db` schemas, including constraints, inserts, and relationships.

2. Migration tests

This verifies that data migration scripts (from legacy schemas to the redesigned schema) preserve integrity and correctness.

The approach guarantees that the schema can be validated repeatedly without side effects, supporting regression testing after each migration.

Chapter 8. Deployment

Deployment scripts focus on reproducibility and automation. All tasks are defined in a `justfile`, which provides a portable command runner for database operations.

This ensures the same steps are executed consistently across developer machines and CI/CD environments.

The environment configuration is specified in `.env.local`:

```
DB_NAME=my_localdb
DB_USER=postgres
DB_HOST=localhost
DB_PORT=5432
PASSWORD=1234
GROUP_NUMBER=12
PROJECT_NAME=portfolio-1

DB_URL_ADMIN=postgresql://${DB_USER}:${PASSWORD}@${DB_HOST}:${DB_PORT}/postgres
DB_URL_TARGET=postgresql://${DB_USER}:${PASSWORD}@${DB_HOST}:${DB_PORT}/${DB_NAME}
```

This setup allows the deployment scripts to use the same commands with different projects, groups, or hosts by adjusting only the environment variables.

Restore from backup

The task `f5-restore-from-backup-no-drop` restores the database into a clean state using the latest backup file.

Steps executed:

1. **Log start of restore** – prints clear markers to the console.
2. **Restore from backup file** – loads the most recent `backup_${DB_NAME}_latest.sql`.
3. **Exit on error** – each command checks `$LASTEXITCODE`, stopping immediately if a step fails.
4. **Log completion** – prints confirmation that the database has been restored.

The deployment process is **deterministic**: running the same task on the same backup file always

Functions would recalculate results every call. Materialized views persist results on disk, can be indexed, and refreshed periodically, making them ideal for heavy analytics.

- `movie_db.get_actor_avg_title_rating` stores weighted averages of actor ratings.

Examples:

Materialized views precompute expensive aggregations that are repeatedly queried.

4.5 Materialized Views

Views behave like tables, can be queried flexibly, and avoid parameter boilerplate.

- `api.get_all_accounts` provides a unified account list.

Examples:

Views expose read-only abstractions that simplify client access.

4.4 Views

Procedures are more suitable for side-effect-driven operations (insert, update, delete) and allow exception handling and notices.

- `api.delete_account` deletes an account and validates existence.
- `api.create_account` inserts a new account and raises a notice with the generated UUID.

Examples:

Stored procedures encapsulate transactional operations where multiple changes must succeed or fail together.

4.3 Stored Procedures